

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PYTHON APLICADO AL ANÁLISIS DE DATOS

Fundamentos de Python

Día 1 · Google Colab, sintaxis básica, tipos de datos, estructuras de control, funciones y módulos

Carlos Jhoan Chiri Maldonado · Taller de Análisis de Datos · UAGRM

ACERCA DE LA SESIÓN

Qué esperar de esta clase

Hoy SÍ haremos

- Escribir y ejecutar código real en Colab
- Pensar problemas de negocio en términos de datos
- Construir bloques que reutilizaremos en las siguientes sesiones
- Equivocarnos en el momento (es parte del aprendizaje)

Hoy NO veremos

- Toda la teoría de programación orientada a objetos
- Pandas y gráficos (los veremos en las siguientes sesiones)
- Modelos de machine learning
- Configuración de entornos locales (usaremos Colab)

Duración: 2h 30min · **Pausa:** 10 min a la mitad · **Formato:** expongo, ejecutamos juntos, ejercicios

PLAN DE LA CLASE

Agenda de hoy

1. Google Colab
15 min · el entorno

2. Sintaxis & módulos
20 min · print, import

3. Tipos de datos
15 min · int, float, str…

4. Estructuras
30 min · listas, dicts, sets

5. Control de flujo
25 min · if, for, while

6. Funciones
20 min · reutilizar lógica

7. NumPy
20 min · el puente a datos

8. Cierre + práctica
15 min · recap y ejercicios

¿POR QUÉ PROGRAMAR?

Del Excel al pensamiento computacional

Tienes las notas de 30 estudiantes en una hoja...
¿Y si fueran 120 en varias materias?

- ¿Qué paralelo o carrera tiene mejor rendimiento?
- ¿Qué variable explica mejor que un estudiante apruebe?
- ¿Cómo se distribuyen los puntajes por materia?

Esto no escala a mano. Necesitamos tres herramientas que trabajen juntas:



Programación

Python



Estadística

conceptos



Análisis

de datos

EL LENGUAJE QUE VAMOS A USAR

¿Por qué Python?

Fácil de leer
Casi inglés. Curva de aprendizaje suave.

Comunidad enorme
Stack Overflow, GitHub, foros, libros…

Ecosistema rico
Pandas, NumPy, scikit-learn, TensorFlow…

Multipropósito
Web, scripts, análisis de datos, IA…

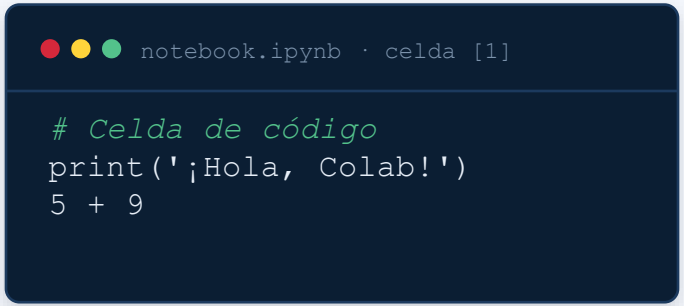
Gratis y portable
El mismo código corre en tu PC y en la nube.

El estándar en datos
Lenguaje #1 en data science a nivel mundial.

Google Colab

Cuadernos (notebooks) de Python que corren en la nube, sin instalar nada.

- **Cero instalación** Solo necesitas una cuenta de Google y un navegador
- **Celdas mixtas** Código (Python) y texto (Markdown) en el mismo documento
- **GPU/TPU gratis** Útil más adelante para modelos pesados
- **Fácil de compartir** Como un documento de Google Drive



Atajos esenciales

Shift + Enter ejecutar y avanzar
Ctrl + Enter ejecutar en el sitio
Alt/Cmd + Enter ejecutar y crear celda

Anatomía de un cuaderno Colab

MD

Análisis de churn
Notas en texto enriquecido (Markdown).

[1]

```
import pandas as pd
df = pd.read_csv('clientes.csv')
```

OUT

```
clientes cargados: 12,480
```

Tres tipos de elementos

- **Texto (Markdown)** Para explicar lo que haces
- **Código (Python)** Lo que se ejecuta al pulsar Run
- **Salida (output)** El resultado de tu celda

💡 *Las celdas se ejecutan en el orden en que tú las corres, no en el orden visual.*

Hola mundo, variables y comentarios

python

```python
# 1. Tu primera línea de Python
print('hello world!')

# 2. Python también es una calculadora
5 + 9

# 3. Guardar un resultado en una variable
total_ventas = 15800
clientes = 240
print(total_ventas, clientes)
```

Ideas clave

`print()` muestra resultados en pantalla

`variable` un nombre que guarda un valor

`nombres` snake_case en minúsculas y sin tildes

`#` comentario: Python lo ignora

`ejecutar` Shift + Enter corre la celda

ARITMÉTICA

Operaciones básicas

Operadores aritméticos

Op	Significado	Ejemplo	Resultado
`+`	suma	`9 + 5`	`14`
`-`	resta	`9 - 5`	`4`
`*`	multiplicación	`9 * 5`	`45`
`**`	potencia	`2 ** 4`	`16`
`/`	división	`10 / 4`	`2.5`
`//`	división entera	`10 // 4`	`2`
`%`	módulo (resto)	`5 % 4`	`1`

calculo_descuento.py

```
# Caso: cálculo rápido de descuento
precio    = 1200
descuento = 0.15

total = precio * (1 - descuento)
ahorro = precio - total

print(total)     # 1020.0
print(ahorro)    # 180.0
```

LÓGICA

Booleanos y comparaciones

booleanos

```
# Comparaciones → True / False
5 > 3          # True
5 == 5         # igualdad (¡doble =!)
5 != 3         # distinto
5 >= 3         # mayor o igual

# Combinar condiciones
5 > 3 and 6 > 3     # True
5 > 3 or  5 < 3     # True
not (5 > 3)         # False
```

¿Para qué sirven?

Son la base de las decisiones del programa.

Ejemplos en negocio:

- ¿Es cliente premium?
- ¿La compra supera los 1000 Bs?
- ¿Tiene más de 6 meses de antigüedad?
- ¿Está en mora?

TIPOS DE DATOS

Cada valor tiene un tipo

int	float	str	bool	None
Entero	Decimal	Texto	Verdadero/Falso	Ausencia de valor
`edad = 25`	`precio = 49.90`	`nombre = 'Ana'`	`activo = True`	`ultima_compra = None`

type · isinstance

```
# Verificar el tipo de cada valor
type(edad), type(precio), type(nombre)     # <class 'int'> <class 'float'> <class
'str'>
isinstance(nombre, str)                    # True  → ¿'nombre' es texto?
```

CAMBIAR DE TIPO

Conversión de tipos (casting)

A veces necesitas convertir un valor de un tipo a otro. Es muy común al leer datos de archivos o formularios.

Expresión	Resultado	Significado
`int('25')`	`25`	texto → entero
`float('49.9')`	`49.9`	texto → decimal
`str(2024)`	`'2024'`	número → texto
`int(3.8)`	`3`	decimal → entero (trunca)
`bool(0)`	`False`	0 es falso
`bool(1)`	`True`	cualquier otro número es verdadero

casting

```
# Caso: dato leído de un CSV viene como texto
monto_str = '1450'
monto = int(monto_str)

# Aplicar descuento del 15%
total = monto * 0.85
print(total)    # 1232.5
```

REUTILIZAR CÓDIGO

Módulos y librerías

python

```python
# 1. Importar un módulo completo
import math
math.sqrt(16)          # 4.0

# 2. Importar funciones específicas
from math import sqrt, cos
sqrt(25)               # 5.0

# 3. Importar con alias (muy común)
import numpy as np

# Ver qué ofrece un módulo
print(dir(math))
```

Los 4 paquetes base del análisis de datos

- **NumPy** cálculo numérico y arrays
- **Pandas** tablas de datos (DataFrames)
- **Matplotlib** gráficos básicos
- **Seaborn** gráficos estadísticos

Listas: colecciones ordenadas

Una lista guarda varios elementos en orden. Es mutable: se puede modificar.

listas

```
# Crear una lista
productos = ['Plan Free', 'Plan 4G', 'Plan 5G']

# Acceder por posición (¡desde 0!)
productos[0]      # 'Plan Free'
productos[-1]     # último: 'Plan 5G'

# Rebanado (slicing)
productos[0:2]    # ['Plan Free', 'Plan 4G']
```

métodos de lista

```
# Modificar
productos.append('Plan Hogar')    # al final
productos.insert(0, 'Prepago')    # al inicio
productos.remove('Plan Free')     # por valor

# Información
len(productos)              # cuántos
'Plan 4G' in productos      # True/False
```

APLICACIÓN PRÁCTICA

Cartera de clientes con listas

CASO DE NEGOCIO Calcular la facturación total de un día — operaciones con listas

ventas_dia.py

```
# Monto de cada venta del día (Bs)
ventas = [120, 80, 450, 60, 230, 90, 1100, 75]

# KPIs básicos
len(ventas)          # cantidad de transacciones
sum(ventas)          # facturación total
max(ventas)          # ticket más alto
min(ventas)          # ticket más bajo
sum(ventas)/len(ventas)    # ticket promedio
```

Salida esperada

Transacciones: 8

Facturación: 2 205 Bs

Ticket máximo: 1 100 Bs

Ticket mínimo: 60 Bs

Ticket promedio: 275.6 Bs

Tuplas y cadenas de texto

Tuplas ( ) inmutables · fijas

tuplas

```
# Coordenadas fijas de una sucursal
sucursal = ('Santa Cruz', -17.78, -63.18)
sucursal[0]       # 'Santa Cruz'
len(sucursal)     # 3

# Desempaquetar en variables
ciudad, lat, lon = sucursal

# No se pueden modificar
# sucursal[0] = 'La Paz'  → ERROR
```

Strings '' texto inmutable

strings

```
cliente = 'Ana Quispe Mamani'

cliente.upper()          # 'ANA QUISPE…'
cliente.lower()          # 'ana quispe…'
cliente.split(' ')       # ['Ana', 'Quispe',…]
len(cliente)             # 18 caracteres
'Quispe' in cliente      # True
cliente.replace('Ana','A.')
```

Diccionarios: clave → valor

Cuando los datos tienen "campos con nombre" (como una fila de una tabla), usa diccionarios.

diccionario · cliente

```
# Ficha de un cliente
cliente = {
    'id': 1001,
    'nombre': 'Ana Quispe',
    'plan': 'Plan 5G',
    'arpu': 95.0,
    'activo': True,
}

cliente['nombre']       # 'Ana Quispe'
cliente['arpu']         # 95.0
```


Operaciones útiles

`.keys()`	lista de claves
`.values()`	lista de valores
`.items()`	pares (clave, valor)
`.get('x', 0)`	valor o default si no existe
`.update({...})`	agregar/actualizar
`len(d)`	cantidad de claves

APLICACIÓN PRÁCTICA

Catálogo de productos con diccionarios

CASO DE NEGOCIO Consultar precios y stock de un catálogo — dict + lookup

catalogo_productos.py

```
# Catálogo: producto → (precio, stock)
catalogo = {
    'PLAN-4G':   (49.0, 850),
    'PLAN-5G':   (89.0, 420),
    'CHIP':      (15.0, 1200),
    'ROUTER':    (350.0, 35),
}

precio, stock = catalogo['PLAN-5G']
print(precio, stock)        # 89.0  420
'ROUTER' in catalogo        # True
```

Cuándo usar dict

Cuando los datos tienen **campos con nombre**

Cuando buscas por **una clave única** (ID, código de producto, NIT)

Para representar una **fila de una tabla** antes de pasar a Pandas.

Conjuntos (sets): elementos únicos

sets

```
# Crear sets
lenguajes = {'Python', 'Java', 'JavaScript'}
skills    = {'Python', 'SQL', 'negocio'}

# Operaciones de teoría de conjuntos
lenguajes | skills    # unión
lenguajes & skills    # intersección
lenguajes - skills    # diferencia

lenguajes.add('Go')
lenguajes.discard('Java')
```

Lo distintivo del set

- **Sin duplicados** agregar lo mismo dos veces no hace nada
- **Sin orden** no puedes pedir set[0]
- **Búsqueda rapidísima** 'x' in set es casi instantáneo
- **Operaciones de conjuntos** unión, intersección, diferencia

APLICACIÓN PRÁCTICA

Clientes en común entre dos canales

CASO DE NEGOCIO ¿Quiénes compran en tienda física Y en e-commerce? operaciones de sets

analisis_canales.py

```
# IDs de clientes que compraron por cada canal este mes
tienda    = {1001, 1002, 1004, 1005, 1010, 1015, 1020}
ecommerce = {1002, 1004, 1011, 1015, 1018, 1020, 1022}

ambos     = tienda & ecommerce        # {1002, 1004, 1015, 1020}  → omnicanal
solo_t    = tienda - ecommerce        # {1001, 1005, 1010}        → solo tienda
solo_e    = ecommerce - tienda        # {1011, 1018, 1022}        → solo online
todos     = tienda | ecommerce        # base total única del mes

tasa_omnicanal = len(ambos) / len(todos)     # 0.40  → 40%
```

Condicionales: if / elif / else

condicionales

```
edad = 25
compras_anio = 4500

if compras_anio > 5000:
    print('Cliente VIP')
elif compras_anio > 2000:
    print('Cliente recurrente')
else:
    print('Cliente nuevo / ocasional')

# Forma corta (ternaria)
estado = 'mayor' if edad >= 18 else 'menor'
```

La indentación importa

En Python el bloque se define con **sangría (4 espacios)**, no con llaves.

El `:` abre el bloque; lo que está indentado debajo pertenece a esa condición.

Colab indenta solo al pulsar Enter dentro del bloque.

APLICACIÓN PRÁCTICA

Segmentación RFM simplificada

CASO DE NEGOCIO Asignar un segmento según consumo y antigüedad — if / elif / else

segmentar_cliente.py

```
consumo_anual = 8200        # Bs gastados en los últimos 12 meses
meses_activo  = 28          # antigüedad
en_mora       = False

if en_mora:
    segmento = 'En riesgo'
elif consumo_anual >= 6000 and meses_activo >= 12:
    segmento = 'VIP'
elif consumo_anual >= 2000:
    segmento = 'Recurrente'
else:
    segmento = 'Nuevo'

print(segmento)     # → 'VIP'
```

Bucle for: recorrer colecciones

for

```
ciudades = ['Santa Cruz', 'La Paz', 'Cbba.']

# Recorrer una lista
for c in ciudades:
    print(c.upper())

# Con índice → enumerate
for i, c in enumerate(ciudades):
    print(i, '→', c)

# Generar números
for n in range(1, 4):
    print(n)              # 1, 2, 3
```

for · dict

```
ventas = {
    'Santa Cruz': 25400,
    'La Paz':     18900,
    'Cbba.':      14200,
}

# Recorrer un diccionario
for ciudad, monto in ventas.items():
    print(ciudad, monto)

# Total con acumulador
total = 0
for v in ventas.values():
    total = total + v
print(total)          # 58 500
```

while, break y continue

while

```
# while: 'mientras se cumpla'
saldo = 1000
intentos = 0

while saldo > 0 and intentos < 3:
    saldo = saldo - 350
    intentos = intentos + 1
    print('saldo:', saldo)

# break  → corta el bucle ahora
# continue → salta a la siguiente vuelta
```

break · continue

```
# Recorrer hasta encontrar al primer moroso
clientes = [
    ('Ana',   False),
    ('Luis',  False),
    ('Mario', True),    # ← moroso
    ('Eva',   False),
]

for nombre, moroso in clientes:
    if not moroso:
        continue       # saltar a la siguiente
    print('alerta:', nombre)
    break              # detener al
encontrarlo
```

APLICACIÓN PRÁCTICA

Reporte mensual por sucursal

CASO DE NEGOCIO Recorrer transacciones y calcular totales por sucursal — for + dict

reporte_sucursales.py

```
transacciones = [
    ('SC-01', 'tienda',    1200),
    ('LP-02', 'ecommerce',  890),
    ('SC-01', 'ecommerce',  450),
    ('CB-03', 'tienda',     320),
    ('LP-02', 'tienda',     780),
    ('SC-01', 'tienda',    1500),
]

total_por_sucursal = {}
for sucursal, canal, monto in transacciones:
    total_por_sucursal[sucursal] = total_por_sucursal.get(sucursal, 0) + monto

print(total_por_sucursal)    # {'SC-01': 3150, 'LP-02': 1670, 'CB-03': 320}
```

ENCAPSULAR LÓGICA

Funciones: bloques reutilizables

funciones

```
# Definir y llamar
def cuadrado(x):
    return x ** 2

cuadrado(5)          # 25

# Argumento con valor por defecto
def aplicar_descuento(monto, pct=0.10):
    return monto * (1 - pct)

aplicar_descuento(1000)         # 900.0
aplicar_descuento(1000, 0.25)   # 750.0
```

Anatomía

def palabra clave para definir

parámetros datos de entrada (x, pct=0.10)

return el valor que devuelve

default valor por si no se pasa

reutilizable se llama tantas veces como quieras

Varios argumentos, varios retornos

argumentos

```python
# Varios argumentos
def precio_final(precio, descuento, iva=0.13):
    sub = precio * (1 - descuento)
    return sub * (1 + iva)

precio_final(1000, 0.1)
# 1017.0

# Documentar la función (docstring)
def saludo(nombre):
    '''Devuelve un saludo personalizado.'''
    return f'Hola, {nombre}!'
```

retornos múltiples

```python
# Devolver varios valores a la vez
def resumen(valores):
    minimo = min(valores)
    maximo = max(valores)
    promedio = sum(valores) / len(valores)
    return minimo, maximo, promedio

ventas = [120, 80, 450, 60, 230, 1100]
mn, mx, prom = resumen(ventas)

print(mn, mx, prom)
# 60  1100  340.0
```

APLICACIÓN PRÁCTICA

Score de cliente con función

CASO DE NEGOCIO Combinar funciones para clasificar clientes — funciones + condicionales

scoring.py

```
def score_cliente(consumo, meses, en_mora=False):
    '''Asigna un score 0-100 según consumo y antigüedad.'''
    if en_mora:
        return 0
    score = 0
    score = score + min(consumo / 100, 60)    # hasta 60 puntos por consumo
    score = score + min(meses * 2, 40)        # hasta 40 puntos por antigüedad
    return round(score, 1)

def clasificar(score):
    return 'VIP' if score >= 80 else 'Recurrente' if score >= 40 else 'Nuevo'

s = score_cliente(consumo=8200, meses=28)    # 88.0
clasificar(s)                                # 'VIP'
```

¿Por qué no bastan las listas?

Con listas normales...

```list
x = [2, 4, 8, 16, 32]
x * 2      # ¡duplica la LISTA!
x.mean()   # ✗ no existe
```

La lista se repite, no se calcula.

Con NumPy...

```numpy
import numpy as np
x = np.array([2, 4, 8, 16, 32])
x * 2          # [4, 8, 16, 32, 64]
x.mean()       # 12.4
np.sqrt(x)     # raíz a cada elemento
x[x > 5]       # filtrado booleano
```

Operaciones sobre todo el array, a la vez (vectorización).

Arrays 2D, indexado y agregaciones

ventas_array.py

```python
import numpy as np

# Ventas (filas = sucursales, cols = trimestres)
ventas = np.array([
    [120, 140, 160, 180],   # SC
    [ 80, 100, 110, 130],   # LP
    [ 60,  70,  80, 100],   # CB
])

ventas.shape          # (3, 4)
ventas[0, :]          # toda la fila SC
ventas[:, -1]         # último trimestre
```

agregaciones

```python
# Agregaciones
ventas.sum()              # total general
ventas.sum(axis=1)        # total por sucursal
ventas.sum(axis=0)        # total por trimestre
ventas.mean(axis=1)       # promedio por
sucursal

# Filtrado booleano
ventas[ventas > 100]
(ventas > 100).sum()      # cuántas celdas

# Crecimiento Q1 → Q4
(ventas[:, 3] - ventas[:, 0]) / ventas[:, 0]
```

APLICACIÓN PRÁCTICA

Análisis de ARPU con NumPy

CASO DE NEGOCIO Ingreso promedio por usuario · churn rate · top cuartil — vectorización

analisis_arpu.py

```
import numpy as np

arpu        = np.array([45, 60, 85, 30, 120, 95, 50, 200, 75, 40])  # Bs/mes por usuario
esta_activo = np.array([1,  1,  1,  0,  1,   1,  0,  1,   1,  0])   # 1=activo, 0=churn

arpu_promedio = arpu.mean()                          # 80.0  Bs
arpu_activos  = arpu[esta_activo == 1].mean()        # 97.1  Bs  (los activos facturan más)
tasa_churn    = (esta_activo == 0).sum() / len(esta_activo)   # 0.30  → 30%

top25 = np.quantile(arpu, 0.75)                      # umbral del top 25%
vip   = arpu[arpu >= top25]                          # clientes en el top cuartil
```

PARA CERRAR

Lo esencial de hoy

- Colab corre Python en la nube con celdas de código y texto
- Tipos: int, float, str, bool, None — y type() / isinstance()
- Estructuras: listas, tuplas, diccionarios y conjuntos
- Control: if/elif/else y bucles for/while con indentación
- Funciones con def/return → y NumPy para calcular sobre datos

Próximos pasos

Día 2: Pandas — tablas reales

Día 3: Matplotlib / Seaborn

Día 4: Estadística aplicada

Practica: rehaz los ejemplos en tu propio Colab.